Indian E-commerce Returns & Fulfilment Intelligence

Exploratory Data Analysis (EDA)

Objective: Analyse item sales, product returns, refunds, and delivery performance to identify useful business insights.

1. Project Overview

- Which product categories generate the most recorded item sales?
- Which categories and reasons are associated with returns?
- How well are delivery promises being met?
- How do sales, returns, and late deliveries change over time?

2. Load and Understand Data

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

DATA_PATH = Path("cleaned_ecommerce_order_items.csv")

if not DATA_PATH.exists():
    raise FileNotFoundError(
        "Cleaned CSV not found. Run
Data_Cleaning_and_Preparation.ipynb first "
        "and keep the exported CSV beside this notebook."
    )

df = pd.read_csv(DATA_PATH)
df.columns = (
    df.columns.str.strip().str.lower()
    .str.replace(r"[^a-z0-9]+", "_", regex=True)
    .str.strip("_")
)

date_cols = ["order_datetime", "shipped_datetime",
"delivered_datetime"]
for col in date_cols:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce")

numeric_cols = [
    "seller_rating", "quantity", "mrp", "discount_pct",
```

```
"selling_price",
    "item_total", "promised_delivery_days", "actual_delivery_days",
    "is_late_delivery", "is_returned", "refund_amount", "refund_days",
    "is_plus_member"
]
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

print("Dataset shape:", df.shape)
display(df.head())
display(df.dtypes.to_frame("Data Type"))
display(df.describe(include="all").T)
```

```
Dataset shape: (165157, 34)
```

```
      order_id  order_item_id      order_datetime customer_id
state  \
0  OD100000001              1 2024-01-01 01:54:00     C015511   Tamil
Nadu
1  OD100000002              1 2024-01-01 08:04:00     C023908
Chandigarh
2  OD100000003              1 2024-01-01 08:26:00     C019249
Maharashtra
3  OD100000004              1 2024-01-01 08:26:00     C022437
Maharashtra
4  OD100000004              2 2024-01-01 08:26:00     C022437
Maharashtra

  region  is_plus_member    payment_method                category  \
0  South               0  Cash on Delivery            Electronics
1  North               1               UPI         Fashion - Women
2   West               0  Cash on Delivery                Footwear
3   West               0            Wallet         Fashion - Women
4   West               0            Wallet  Beauty & Personal Care

         sub_category  ... is_late_delivery        order_status
cancel_reason  \
0             Laptops  ...              0.0           Delivered
NaN
1      Kurtas & Suits  ...              0.0           Delivered
NaN
2  Sandals & Floaters  ...              0.0            Returned
NaN
3         Ethnic Sets  ...              0.0  Partially Returned
NaN
4              Makeup  ...              0.0  Partially Returned
NaN

   item_status  is_returned           return_reason  return_status  \
```

```
0    Delivered            0                       NaN            NaN
1    Delivered            0                       NaN            NaN
2     Returned            1  Quality not as expected       Approved
3    Delivered            0                       NaN            NaN
4    Delivered            0                       NaN            NaN

     refund_mode  refund_amount refund_days
0            NaN            NaN         NaN
1            NaN            NaN         NaN
2  Bank Transfer          709.0         8.0
3            NaN            NaN         NaN
4            NaN            NaN         NaN

[5 rows x 34 columns]
```

	Data Type
order_id	str
order_item_id	int64
order_datetime	datetime64[us]
customer_id	str
state	str
region	str
is_plus_member	int64
payment_method	str
category	str
sub_category	str
brand	str
product_id	str
seller_id	str
seller_rating	float64
quantity	int64
mrp	float64
discount_pct	float64
selling_price	float64
item_total	float64
courier_partner	str
promised_delivery_days	int64
shipped_datetime	datetime64[us]
delivered_datetime	datetime64[us]
actual_delivery_days	float64
is_late_delivery	float64
order_status	str
cancel_reason	str
item_status	str
is_returned	int64
return_reason	str
return_status	str
refund_mode	str
refund_amount	float64
refund_days	float64

	count	unique	top	freq
order_id	165157	90000	OD100000999	5
order_item_id	165157.0	NaN	NaN	NaN
order_datetime	165157	NaN	NaN	NaN
customer_id	165157	19892	C005234	243
state	165157	18	Maharashtra	30314
region	165157	6	South	57634
is_plus_member	165157.0	NaN	NaN	NaN
payment_method	165157	7	UPI	81584
category	165157	13	Fashion - Women	22837
sub_category	165157	65	Feature Phones	8149
brand	165157	198	Krika	3255
product_id	165157	1799	P01496	2754
seller_id	165157	397	S0231	3158
seller_rating	165157.0	NaN	NaN	NaN
quantity	165157.0	NaN	NaN	NaN
mrp	165157.0	NaN	NaN	NaN
discount_pct	165157.0	NaN	NaN	NaN
selling_price	165157.0	NaN	NaN	NaN
item_total	165157.0	NaN	NaN	NaN
courier_partner	165157	4	Courier A	49863
promised_delivery_days	165157.0	NaN	NaN	NaN
shipped_datetime	158141	NaN	NaN	NaN
delivered_datetime	156304	NaN	NaN	NaN
actual_delivery_days	156304.0	NaN	NaN	NaN

```
is_late_delivery        156304.0    NaN                       NaN
NaN
order_status              165157      5                 Delivered
113399
cancel_reason               7016      5  Found cheaper elsewhere
1797
item_status               165157      4                 Delivered
135461
is_returned             165157.0    NaN                       NaN
NaN
return_reason             20843     11  Quality not as expected
4013
return_status             20843      2                  Approved
19771
refund_mode               20843      3  Original Payment Method
14767
refund_amount            20843.0    NaN                       NaN
NaN
refund_days              19771.0    NaN                       NaN
NaN

                                                mean
min  \
order_id                                         NaN
NaN
order_item_id                               1.720236
1.0
order_datetime          2025-07-06 14:47:19.360850  2024-01-01
01:54:00
customer_id                                      NaN
NaN
state                                            NaN
NaN
region                                           NaN
NaN
is_plus_member                              0.159497
0.0
payment_method                                   NaN
NaN
category                                         NaN
NaN
sub_category                                     NaN
NaN
brand                                            NaN
NaN
product_id                                       NaN
NaN
seller_id                                        NaN
NaN
seller_rating                               4.064754
```

```
2.8
quantity                                    1.307598
1.0
mrp                                      3505.532572
99.0
discount_pct                               30.699679
0.0
selling_price                            2803.742942
49.0
item_total                               3391.756032
49.0
courier_partner                                  NaN
NaN
promised_delivery_days                     2.574556
1.0
shipped_datetime        2025-07-07 21:56:17.161394  2024-01-01
18:51:00
delivered_datetime      2025-07-10 20:34:02.323933  2024-01-02
10:20:00
actual_delivery_days                       2.792149
1.0
is_late_delivery                           0.252521
0.0
order_status                                     NaN
NaN
cancel_reason                                    NaN
NaN
item_status                                      NaN
NaN
is_returned                                0.126201
0.0
return_reason                                    NaN
NaN
return_status                                    NaN
NaN
refund_mode                                      NaN
NaN
refund_amount                            2082.136833
0.0
refund_days                                4.985888
2.0

                                            25%                  50%  \
order_id                                    NaN                  NaN
order_item_id                               1.0                  1.0
order_datetime          2024-11-13 16:08:00  2025-08-13 19:55:00
customer_id                                 NaN                  NaN
state                                       NaN                  NaN
region                                      NaN                  NaN
is_plus_member                              0.0                  0.0
```

payment_method		NaN	NaN	
category		NaN	NaN	
sub_category		NaN	NaN	
brand		NaN	NaN	
product_id		NaN	NaN	
seller_id		NaN	NaN	
seller_rating		3.8	4.1	
quantity		1.0	1.0	
mrp		489.0	869.0	
discount_pct		19.8	30.5	
selling_price		316.0	541.0	
item_total		382.0	662.0	
courier_partner		NaN	NaN	
promised_delivery_days		2.0	2.0	
shipped_datetime	2024-11-15 02:58:00	2025-08-15 01:16:00		
delivered_datetime	2024-11-18 19:25:30	2025-08-18 13:42:00		
actual_delivery_days		2.0	2.0	
is_late_delivery		0.0	0.0	
order_status		NaN	NaN	
cancel_reason		NaN	NaN	
item_status		NaN	NaN	
is_returned		0.0	0.0	
return_reason		NaN	NaN	
return_status		NaN	NaN	
refund_mode		NaN	NaN	
refund_amount		348.5	594.0	
refund_days		3.0	5.0	

		75%	max
std			
order_id		NaN	NaN
NaN			
order_item_id		2.0	5.0
0.9381			
order_datetime	2026-02-15 19:55:00	2026-09-30 23:50:00	
NaN			
customer_id		NaN	NaN
NaN			
state		NaN	NaN
NaN			
region		NaN	NaN
NaN			
is_plus_member		0.0	1.0
0.36614			
payment_method		NaN	NaN
NaN			
category		NaN	NaN
NaN			
sub_category		NaN	NaN
NaN			

```
brand                                  NaN                  NaN
NaN
product_id                             NaN                  NaN
NaN
seller_id                              NaN                  NaN
NaN
seller_rating                          4.3                  4.9
0.334537
quantity                               2.0                  3.0
0.534092
mrp                                 1619.0               104679.0
8508.117637
discount_pct                          41.5                 85.0
15.09268
selling_price                       1029.0               104679.0
7323.780969
item_total                          1326.0               209358.0
9270.66504
courier_partner                        NaN                  NaN
NaN
promised_delivery_days                 3.0                  5.0
1.143395
shipped_datetime        2026-02-17 22:13:00  2026-10-02 05:48:00
NaN
delivered_datetime      2026-02-20 16:29:00  2026-10-07 20:52:00
NaN
actual_delivery_days                   3.0                 10.0
1.363466
is_late_delivery                       1.0                  1.0
0.43446
order_status                           NaN                  NaN
NaN
cancel_reason                          NaN                  NaN
NaN
item_status                            NaN                  NaN
NaN
is_returned                            0.0                  1.0
0.332077
return_reason                          NaN                  NaN
NaN
return_status                          NaN                  NaN
NaN
refund_mode                            NaN                  NaN
NaN
refund_amount                       1030.0               127144.0
6985.417719
refund_days                            7.0                  8.0
2.004914
```

3. Business KPIs

```
total_orders = df["order_id"].nunique() if "order_id" in df.columns
else 0
total_items = len(df)
recorded_item_sales = df["item_total"].sum(min_count=1) if
"item_total" in df.columns else float("nan")

# Average recorded order value: sum item_total within each order, then
average across orders.
if {"order_id", "item_total"}.issubset(df.columns):
    order_values = df.groupby("order_id", dropna=True)
["item_total"].sum(min_count=1)
    average_order_value = order_values.mean()
else:
    average_order_value = float("nan")

if "is_returned" in df.columns:
    returned = df["is_returned"].eq(1)
    known_returns = df["is_returned"].isin([0, 1])
    returned_count = int(returned.sum())
    return_rate = (returned.sum() / known_returns.sum() * 100) if
known_returns.sum() else float("nan")
else:
    returned_count, return_rate = 0, float("nan")

if "is_late_delivery" in df.columns:
    known_late = df["is_late_delivery"].isin([0, 1])
    late_count = int(df["is_late_delivery"].eq(1).sum())
    late_rate = (late_count / known_late.sum() * 100) if
known_late.sum() else float("nan")
else:
    late_count, late_rate = 0, float("nan")

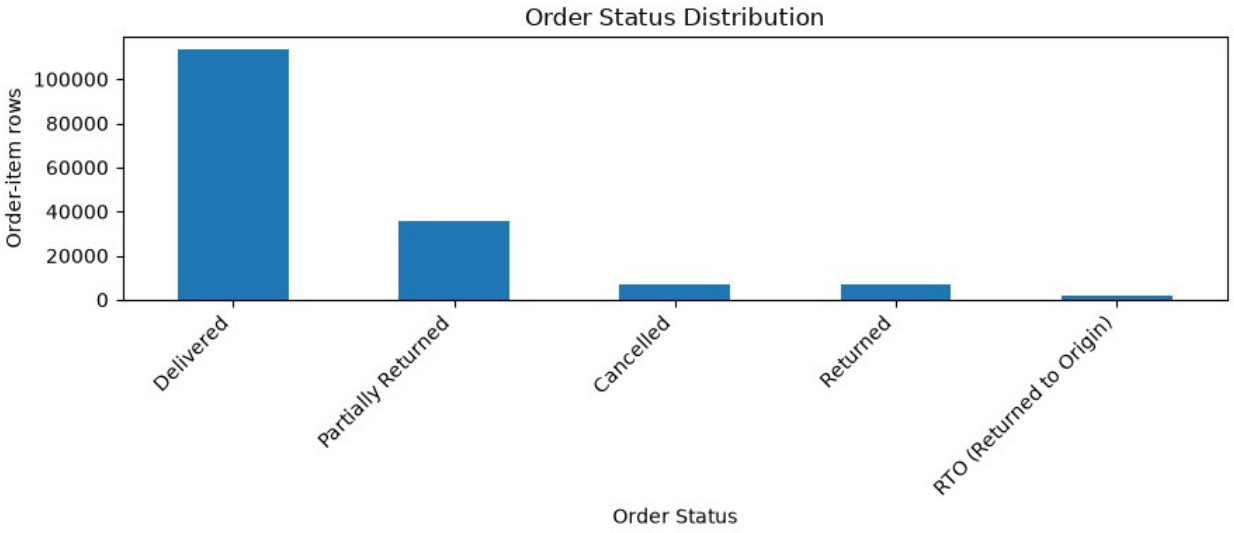
kpis = pd.DataFrame({
    "Metric": [
        "Total Orders", "Total Order Items", "Recorded Item Sales",
        "Average Recorded Order Value", "Returned Items", "Return Rate
(%)",
        "Late Delivery Items with Known Flag", "Late Delivery Rate
(%)"
    ],
    "Value": [
        total_orders, total_items, recorded_item_sales,
average_order_value,
        returned_count, return_rate, late_count, late_rate
    ]
})
display(kpis)
```

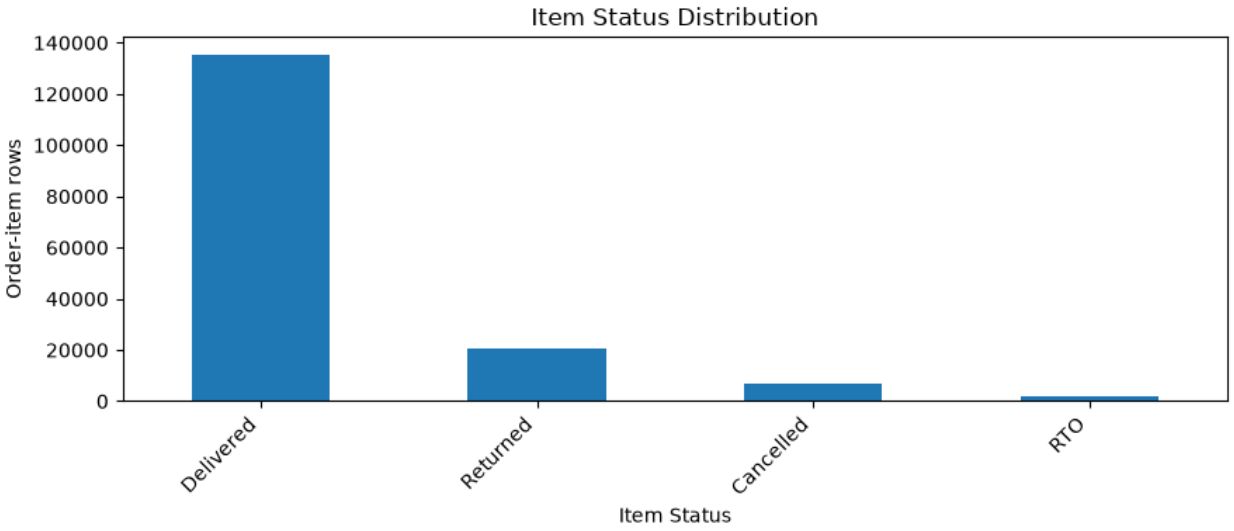
```
                                Metric         Value
0                         Total Orders  9.000000e+04
1                   Total Order Items  1.651570e+05
2                  Recorded Item Sales  5.601723e+08
3        Average Recorded Order Value  6.224136e+03
4                      Returned Items  2.084300e+04
5                    Return Rate (%)  1.262011e+01
6  Late Delivery Items with Known Flag  3.947000e+04
7              Late Delivery Rate (%)  2.525207e+01
```

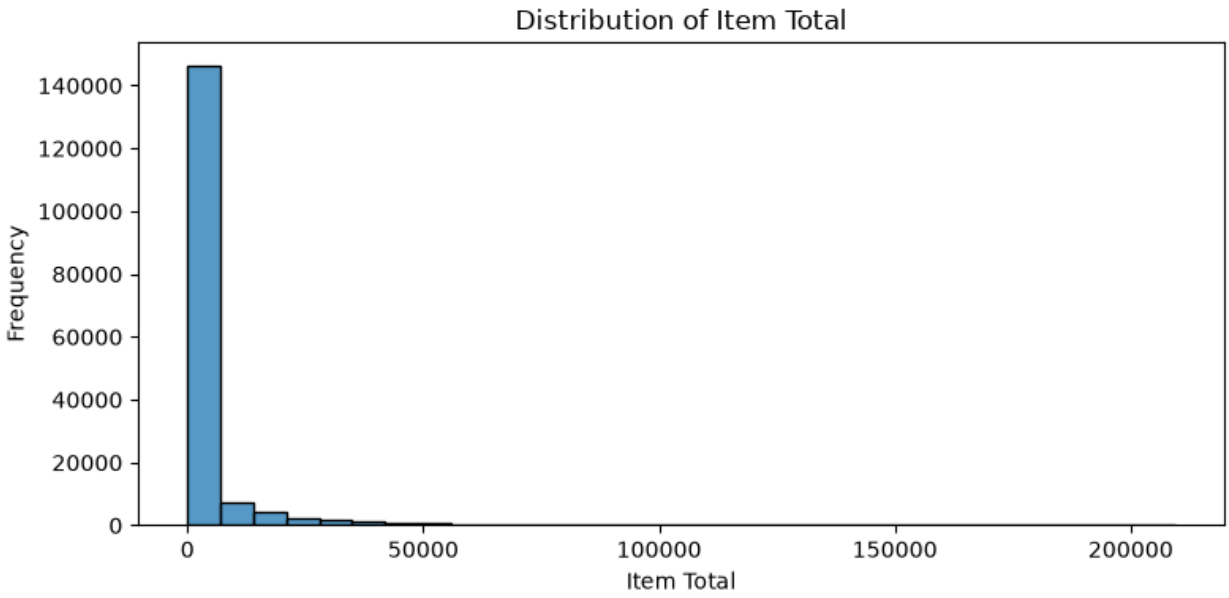
4. Univariate Analysis

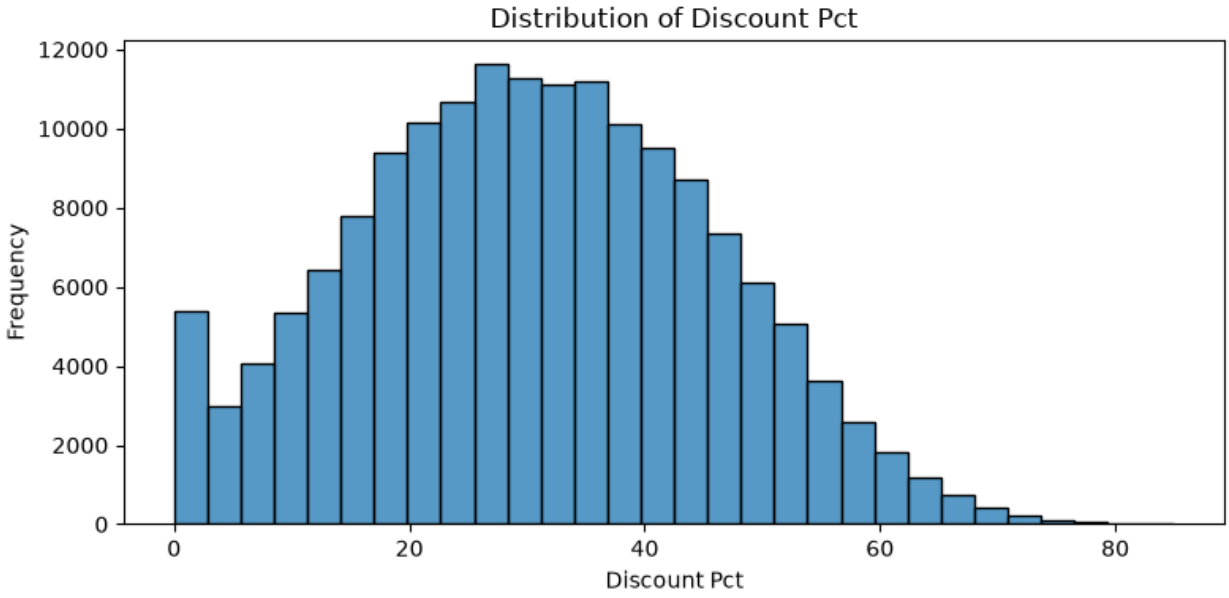
```
# Categorical distributions
for col in ["order_status", "item_status"]:
    if col in df.columns and df[col].notna().any():
        counts = df[col].fillna("Missing").value_counts()
        ax = counts.plot(kind="bar", figsize=(9, 4),
title=f"{col.replace('_', ' ').title()} Distribution")
        ax.set_xlabel(col.replace("_", " ").title())
        ax.set_ylabel("Order-item rows")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()

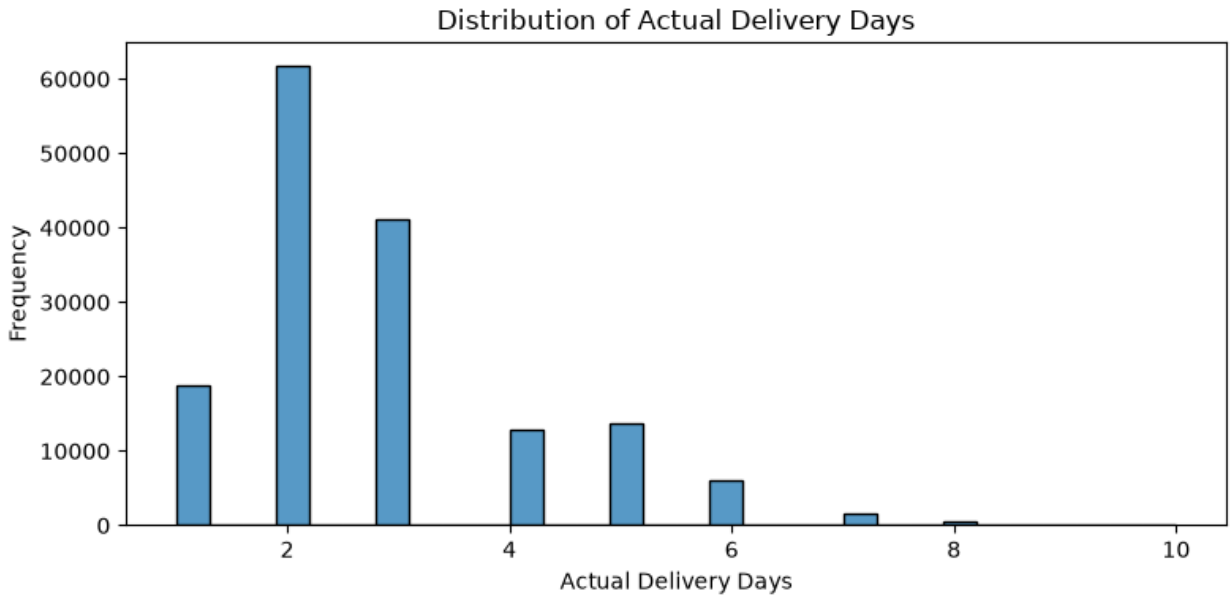
# Numeric distributions
for col in ["item_total", "discount_pct", "actual_delivery_days"]:
    if col in df.columns and df[col].notna().any():
        plt.figure(figsize=(8, 4))
        sns.histplot(df[col].dropna(), bins=30, kde=False)
        plt.title(f"Distribution of {col.replace('_', ' ').title()}")
        plt.xlabel(col.replace("_", " ").title())
        plt.ylabel("Frequency")
        plt.tight_layout()
        plt.show()
```


Order Status Distribution
Order-item rows
100000
80000
60000
40000
20000
0
Delivered
Partially Returned
Cancelled
Returned
RTO (Returned to Origin)
Order Status


Item Status Distribution
Order-item rows
140000
120000
100000
80000
60000
40000
20000
0
Delivered
Returned
Cancelled
RTO
Item Status


Distribution of Item Total
Frequency
140000
120000
100000
80000
60000
40000
20000
0
0
50000
100000
150000
200000
Item Total


Distribution of Discount Pct
Frequency
12000
10000
8000
6000
4000
2000
0
0
20
40
60
80
Discount Pct



5. Bivariate Analysis

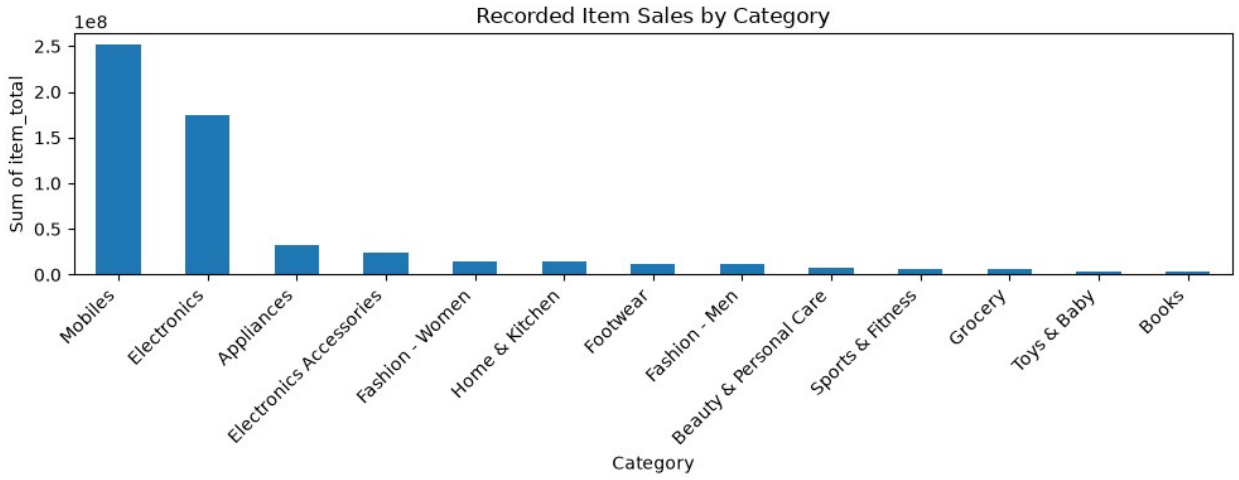
```
# Recorded item sales by product category
if {"category", "item_total"}.issubset(df.columns):
    category_sales = df.groupby("category", dropna=False)
["item_total"].sum().sort_values(ascending=False)
    display(category_sales.to_frame("Recorded Item Sales"))
    category_sales.plot(kind="bar", figsize=(10, 4), title="Recorded
Item Sales by Category")
    plt.xlabel("Category")
    plt.ylabel("Sum of item_total")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

# Return rate by category; exclude unknown return flags from the
denominator
if {"category", "is_returned"}.issubset(df.columns):
    known = df[df["is_returned"].isin([0, 1])]
    if not known.empty:
        category_return = known.groupby("category", dropna=False)
["is_returned"].mean().mul(100).sort_values(ascending=False)
        display(category_return.to_frame("Return Rate (%)"))
        category_return.plot(kind="bar", figsize=(10, 4),
title="Return Rate by Category")
        plt.ylabel("Return Rate (%)")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()

# Discount vs item sales
```

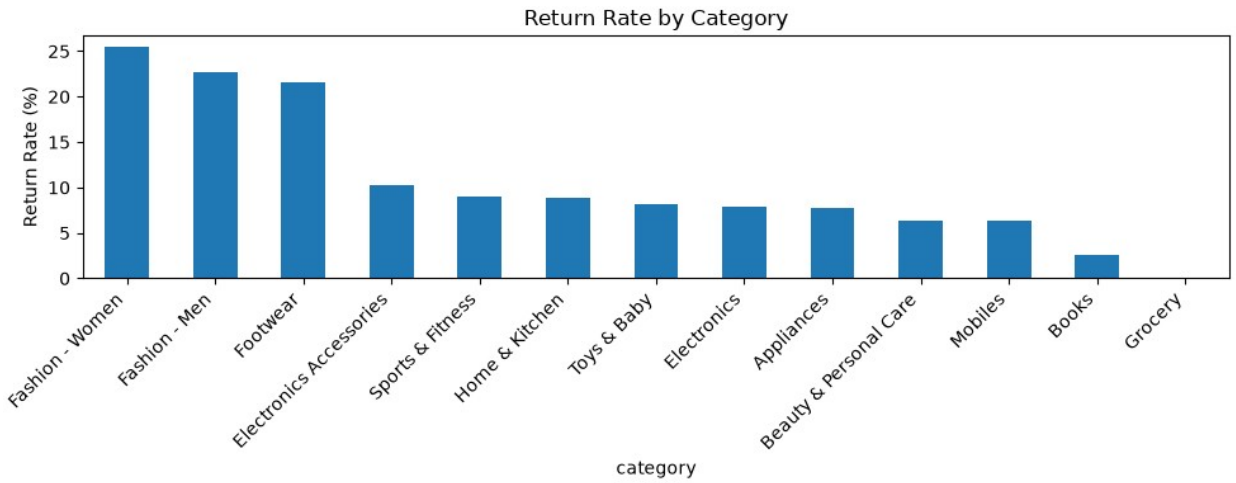
```
if {"discount_pct", "item_total"}.issubset(df.columns):
    plot_data = df[["discount_pct", "item_total"]].dropna()
    if not plot_data.empty:
        plt.figure(figsize=(7, 4))
        sns.scatterplot(data=plot_data, x="discount_pct",
y="item_total", alpha=0.35)
        plt.title("Discount Percentage vs. Item Total")
        plt.tight_layout()
        plt.show()
```

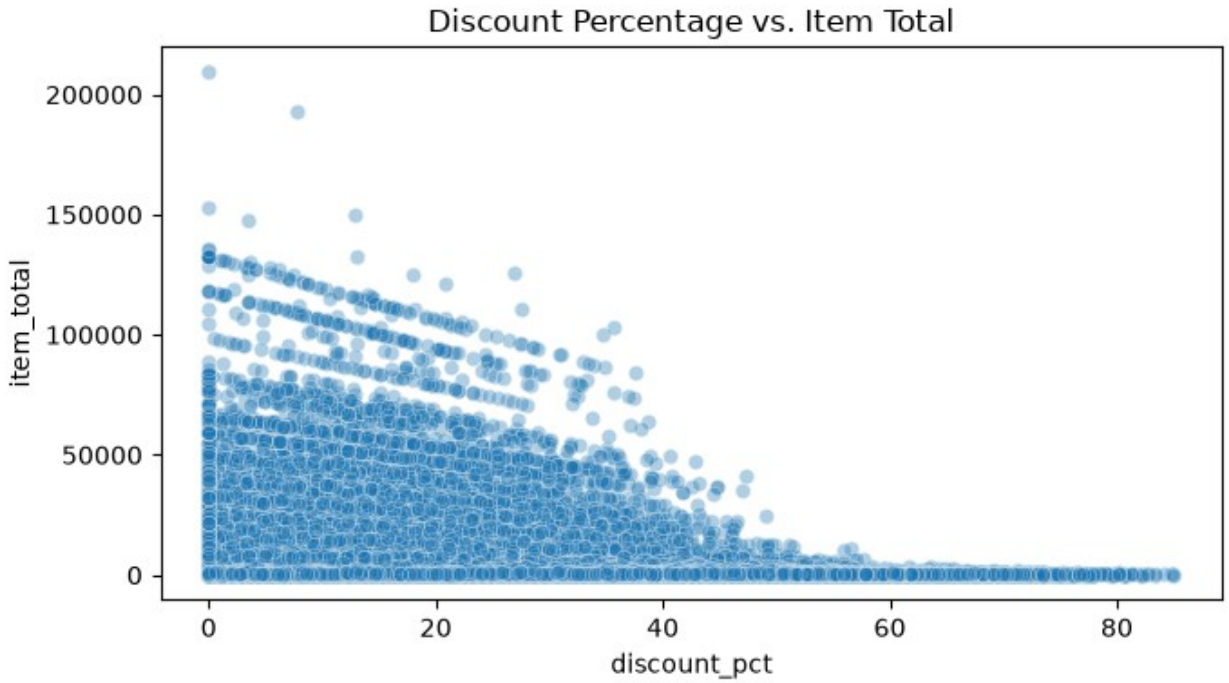
category	Recorded Item Sales
Mobiles	251238157.0
Electronics	174090514.0
Appliances	32931773.0
Electronics Accessories	23665529.0
Fashion - Women	14889011.0
Home & Kitchen	14555455.0
Footwear	12198606.0
Fashion - Men	11191249.0
Beauty & Personal Care	7052558.0
Sports & Fitness	6223101.0
Grocery	5779222.0
Toys & Baby	3231420.0
Books	3125656.0



category	Return Rate (%)
Fashion - Women	25.428033
Fashion - Men	22.700722
Footwear	21.602938
Electronics Accessories	10.297553
Sports & Fitness	9.007732
Home & Kitchen	8.869323

```
Toys & Baby                    8.133765
Electronics                    7.847484
Appliances                     7.762729
Beauty & Personal Care         6.352346
Mobiles                        6.305347
Books                          2.584731
Grocery                        0.000000
```





6. Multivariate Analysis

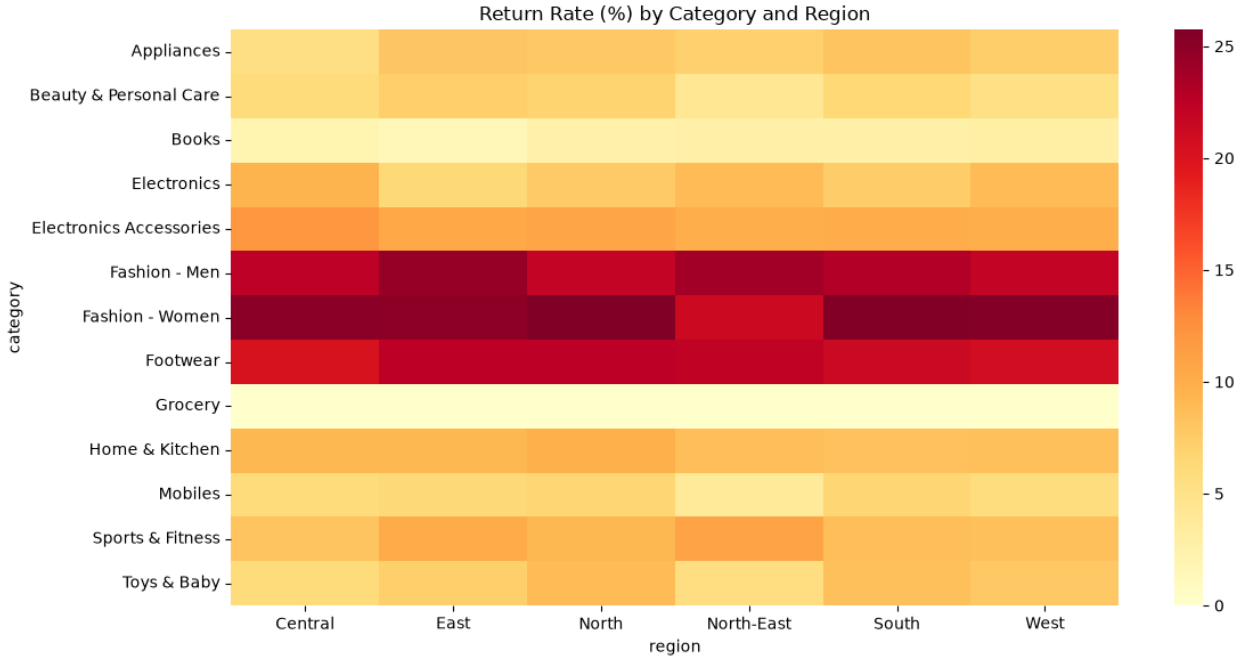
```
# Return rate across category and region
if {"category", "region", "is_returned"}.issubset(df.columns):
    known = df[df["is_returned"].isin([0, 1])]
```

```
    if not known.empty:
        return_pivot = known.pivot_table(
            index="category", columns="region", values="is_returned",
            aggfunc="mean"
        ) * 100
        display(return_pivot.round(2))
        plt.figure(figsize=(12, 6))
        sns.heatmap(return_pivot, annot=False, cmap="YlOrRd")
        plt.title("Return Rate (%) by Category and Region")
        plt.tight_layout()
        plt.show()

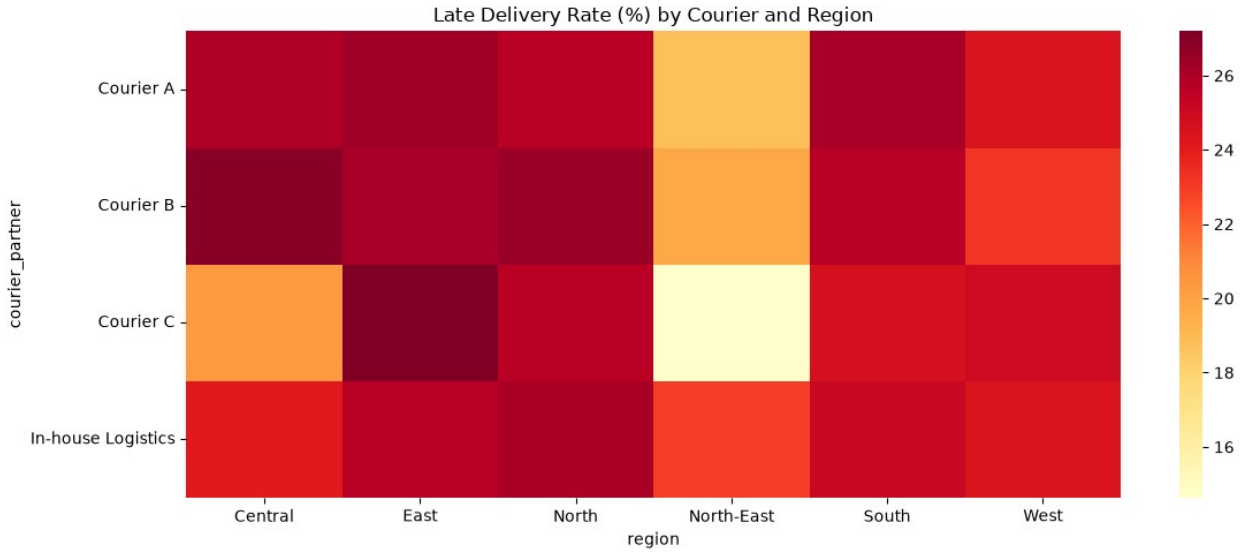
# Late delivery rate by courier and region
if {"courier_partner", "region",
"is_late_delivery"}.issubset(df.columns):
    known = df[df["is_late_delivery"].isin([0, 1])]
    if not known.empty:
        late_pivot = known.pivot_table(
            index="courier_partner", columns="region",
values="is_late_delivery",
            aggfunc="mean"
        ) * 100
        display(late_pivot.round(2))
        plt.figure(figsize=(12, 5))
        sns.heatmap(late_pivot, annot=False, cmap="YlOrRd")
        plt.title("Late Delivery Rate (%) by Courier and Region")
        plt.tight_layout()
        plt.show()
```

region	Central	East	North	North-East	South	West
category						
Appliances	5.43	8.04	7.77	7.09	8.16	7.38
Beauty & Personal Care	5.99	7.28	6.90	4.29	6.53	5.29
Books	2.01	1.60	2.57	2.73	2.75	2.91
Electronics	9.45	6.36	7.71	8.86	7.51	8.90
Electronics Accessories	11.86	10.53	10.75	10.00	10.06	9.94
Fashion - Men	22.39	24.45	21.94	23.84	23.10	21.96
Fashion - Women	25.16	25.03	25.73	21.23	25.58	25.45
Footwear	20.21	22.51	22.42	22.27	21.37	20.84
Grocery	0.00	0.00	0.00	0.00	0.00	

```
0.00
Home & Kitchen              9.23   9.20   9.77        8.68   8.42
8.51
Mobiles                     6.03   6.35   6.61        3.77   6.58
5.83
Sports & Fitness            8.20  10.26   9.21       10.95   8.73
8.52
Toys & Baby                 5.93   7.14   8.90        5.71   8.49
7.83
```



region	Central	East	North	North-East	South	West
courier_partner						
Courier A	25.95	26.35	25.64	18.80	26.14	24.49
Courier B	26.97	26.13	26.47	19.72	25.69	23.21
Courier C	20.36	27.20	25.65	14.63	24.58	24.96
In-house Logistics	24.10	25.67	26.08	22.95	25.10	24.44



7. Correlation Analysis

```
corr_cols = [
    col for col in [
        "quantity", "mrp", "discount_pct", "selling_price",
"item_total",
        "seller_rating", "promised_delivery_days",
"actual_delivery_days",
        "refund_amount", "refund_days"
    ]
    if col in df.columns and pd.api.types.is_numeric_dtype(df[col])
]

corr_data = df[corr_cols].dropna(how="all") if corr_cols else
pd.DataFrame()
if len(corr_cols) >= 2 and not corr_data.empty:
    correlation = corr_data.corr(numeric_only=True)
    display(correlation.round(2))
    plt.figure(figsize=(10, 7))
    sns.heatmap(correlation, annot=True, fmt=".2f", cmap="coolwarm",
center=0)
    plt.title("Correlation Heatmap")
    plt.tight_layout()
    plt.show()
else:
    print("Not enough numeric columns with data for correlation
analysis.")

# Correlation shows association, not causation.
```

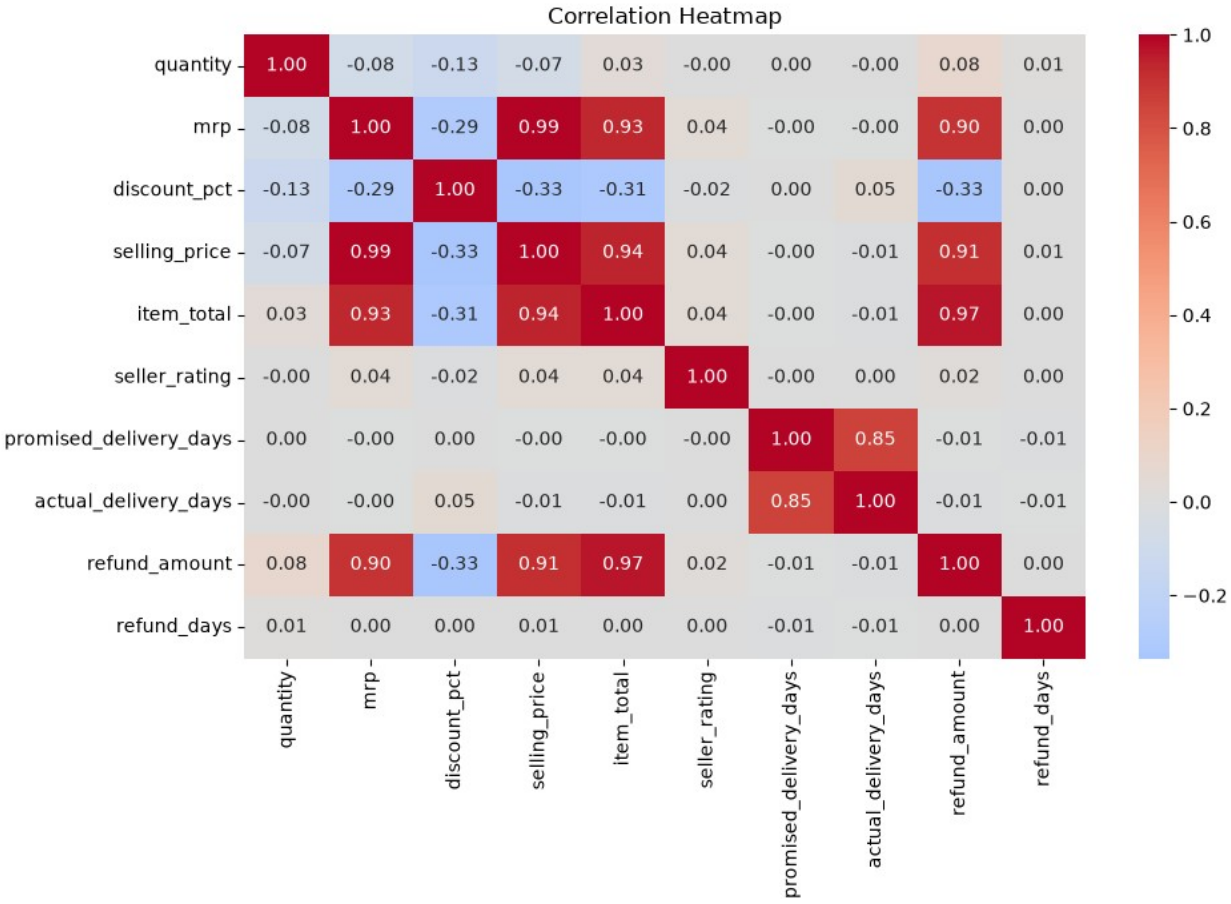
```
                        quantity   mrp  discount_pct  selling_price  \
quantity                    1.00 -0.08         -0.13          -0.07
mrp                        -0.08  1.00         -0.29           0.99
```

```
discount_pct                -0.13 -0.29          1.00          -0.33
selling_price               -0.07  0.99         -0.33           1.00
item_total                   0.03  0.93         -0.31           0.94
seller_rating               -0.00  0.04         -0.02           0.04
promised_delivery_days       0.00 -0.00          0.00          -0.00
actual_delivery_days        -0.00 -0.00          0.05          -0.01
refund_amount                0.08  0.90         -0.33           0.91
refund_days                  0.01  0.00          0.00           0.01

                         item_total  seller_rating
promised_delivery_days  \
quantity                       0.03          -0.00
0.00
mrp                            0.93           0.04                    -
0.00
discount_pct                  -0.31          -0.02
0.00
selling_price                  0.94           0.04                    -
0.00
item_total                     1.00           0.04                    -
0.00
seller_rating                  0.04           1.00                    -
0.00
promised_delivery_days        -0.00          -0.00
1.00
actual_delivery_days          -0.01           0.00
0.85
refund_amount                  0.97           0.02                    -
0.01
refund_days                    0.00           0.00                    -
0.01

                         actual_delivery_days  refund_amount
refund_days
quantity                                -0.00           0.08
0.01
mrp                                     -0.00           0.90
0.00
discount_pct                             0.05          -0.33
0.00
selling_price                           -0.01           0.91
0.01
item_total                              -0.01           0.97
0.00
seller_rating                            0.00           0.02
0.00
promised_delivery_days                   0.85          -0.01         -
0.01
actual_delivery_days                     1.00          -0.01         -
```

```
0.01
refund_amount                          -0.01                1.00
0.00
refund_days                            -0.01                0.00
1.00
```



8. Outlier Analysis

```python
outlier_cols = ["item_total", "selling_price", "actual_delivery_days"]

for col in outlier_cols:
    if col not in df.columns:
        continue
    values = df[col].dropna()
    if values.empty:
        continue

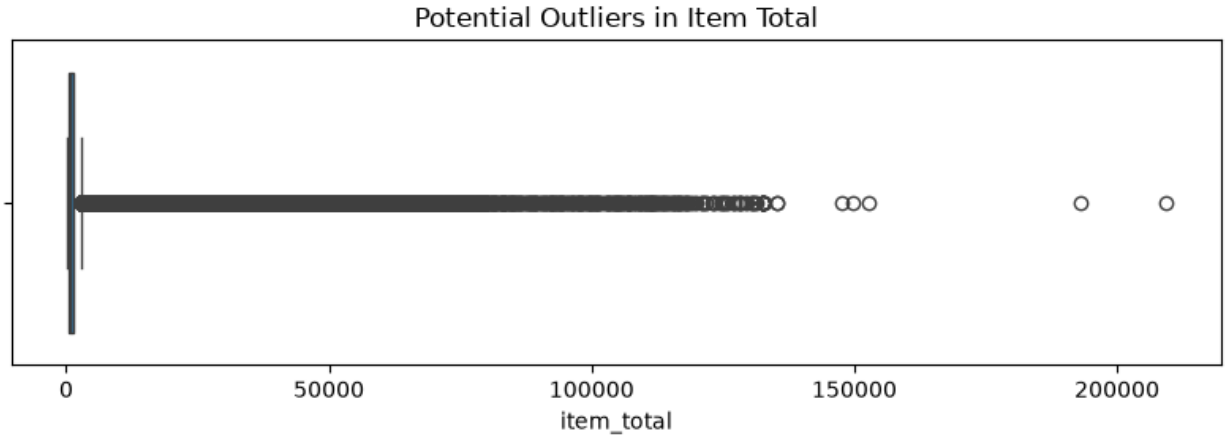
    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
```

```
    outlier_count = int(((values < lower) | (values > upper)).sum())

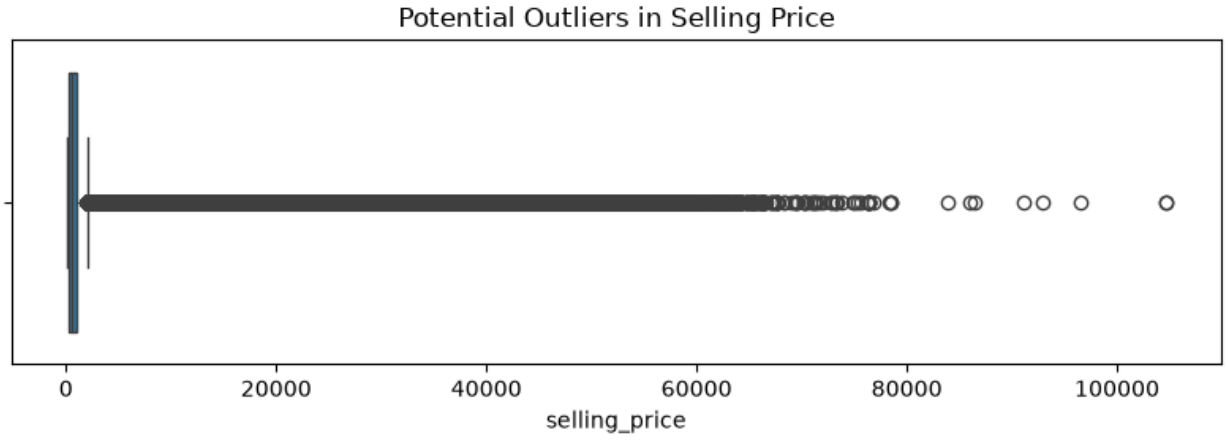
    print(f"{col}: {outlier_count:,} potential outliers (IQR method)")
    plt.figure(figsize=(8, 3))
    sns.boxplot(x=values)
    plt.title(f"Potential Outliers in {col.replace('_', '
').title()}")
    plt.tight_layout()
    plt.show()

# Outliers are flagged for review, not automatically removed.
```

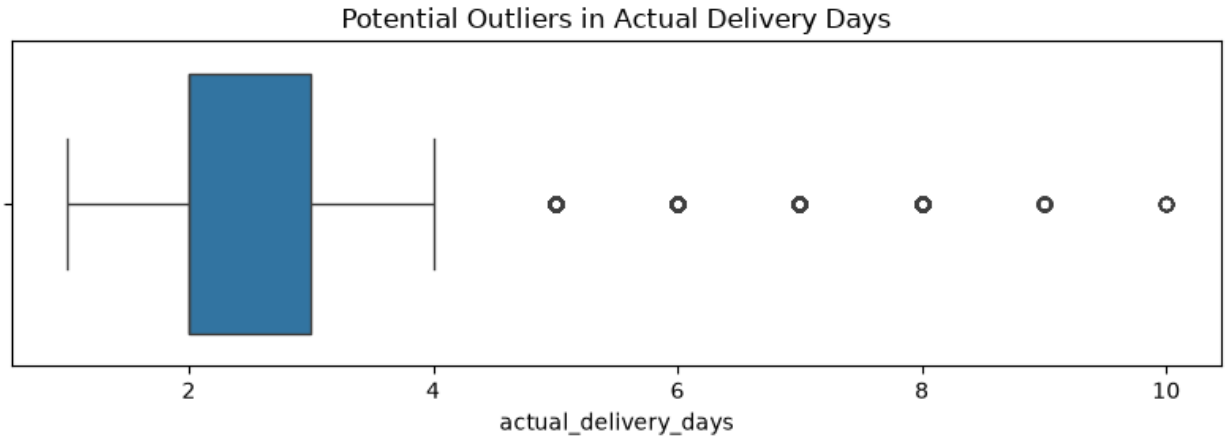
```
item_total: 26,219 potential outliers (IQR method)
```




```
selling_price: 27,417 potential outliers (IQR method)
```




```
actual_delivery_days: 21,713 potential outliers (IQR method)
```



9. Product and Return Analysis

```
# Sales by sub-category
if {"sub_category", "item_total"}.issubset(df.columns):
    subcategory_sales = df.groupby("sub_category", dropna=False)
["item_total"].sum().sort_values(ascending=False).head(15)
    display(subcategory_sales.to_frame("Recorded Item Sales"))
    subcategory_sales.sort_values().plot(kind="barh", figsize=(9, 5),
title="Top 15 Sub-categories by Recorded Item Sales")
    plt.xlabel("Sum of item_total")
    plt.tight_layout()
    plt.show()

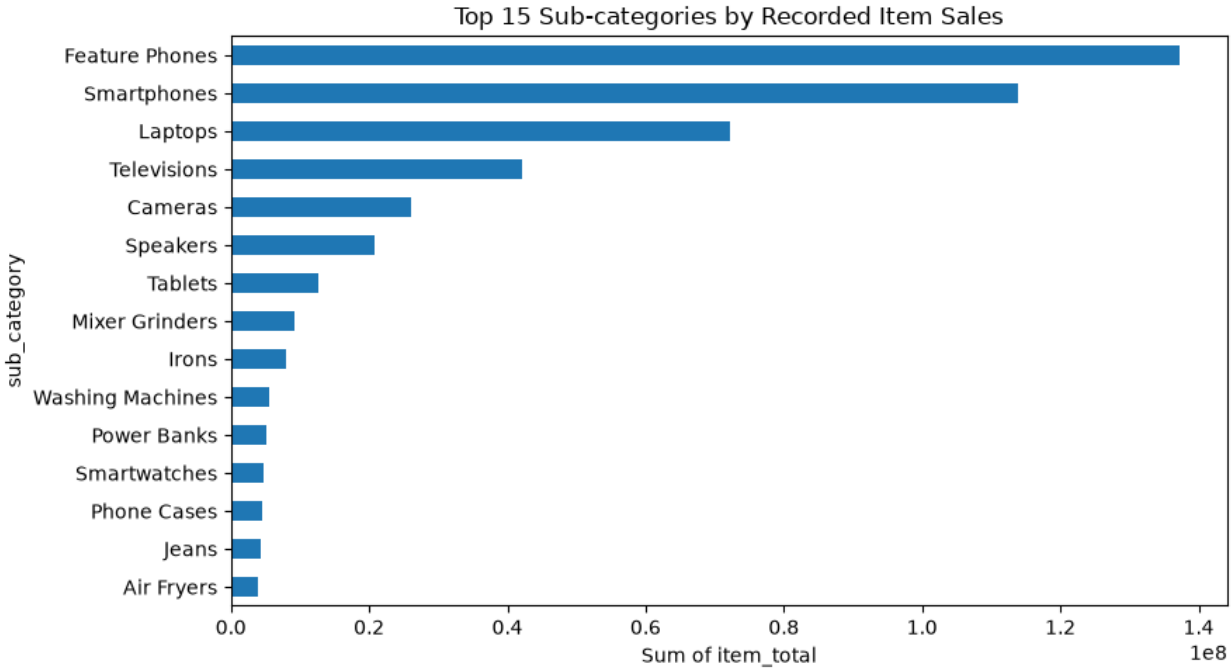
# Return reasons and statuses
for col in ["return_reason", "return_status"]:
    if col in df.columns:
        counts = df[col].dropna().value_counts()
        if not counts.empty:
            display(counts.to_frame("Order-item Rows"))
            counts.head(10).plot(kind="bar", figsize=(8, 4),
title=f"{col.replace('_', ' ').title()} Distribution")
            plt.ylabel("Order-item rows")
            plt.xticks(rotation=45, ha="right")
            plt.tight_layout()
            plt.show()

# Recorded refunds: values are not confirmation of refund settlement
if "refund_amount" in df.columns:
    refund_rows = df[df["refund_amount"].notna()]
    print("Rows with a recorded refund amount:", len(refund_rows))
    print("Total recorded refund amount:",
round(refund_rows["refund_amount"].sum(), 2))
    if "refund_mode" in refund_rows.columns:

display(refund_rows["refund_mode"].value_counts(dropna=False).to_frame
("Rows"))
```

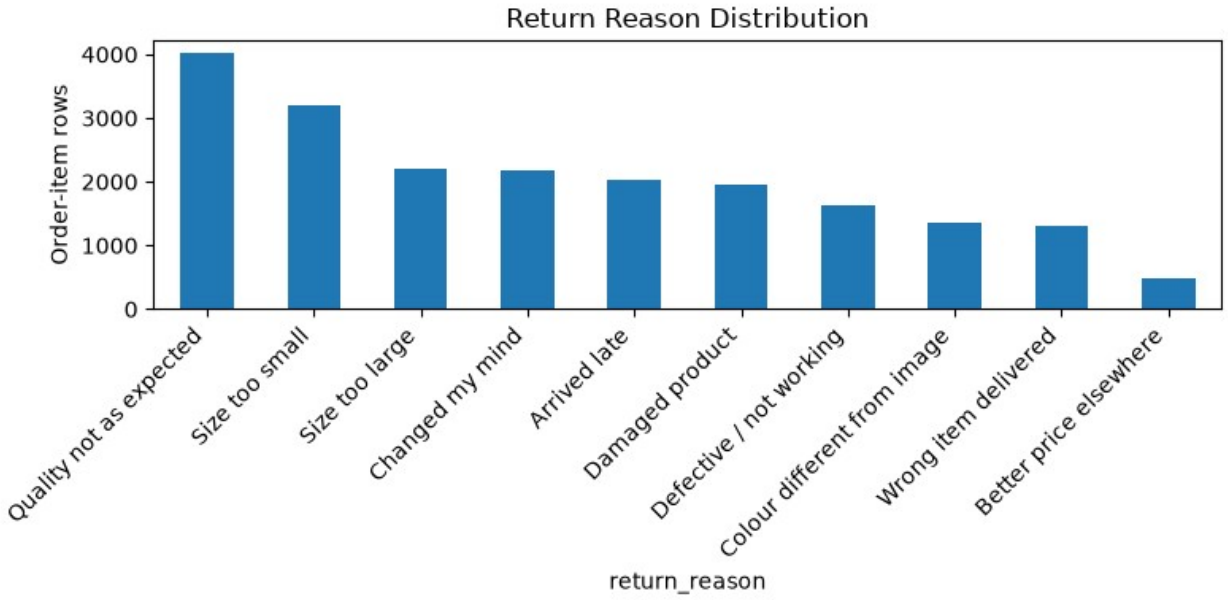
```
    if "refund_days" in refund_rows.columns:
        print("Average recorded refund days:",
round(refund_rows["refund_days"].mean(), 2))
```

sub_category	Recorded Item Sales
Feature Phones	137301327.0
Smartphones	113936830.0
Laptops	72192309.0
Televisions	42210582.0
Cameras	26157811.0
Speakers	20827276.0
Tablets	12702536.0
Mixer Grinders	9287820.0
Irons	7950032.0
Washing Machines	5572045.0
Power Banks	5059040.0
Smartwatches	4816524.0
Phone Cases	4437400.0
Jeans	4286696.0
Air Fryers	4005515.0

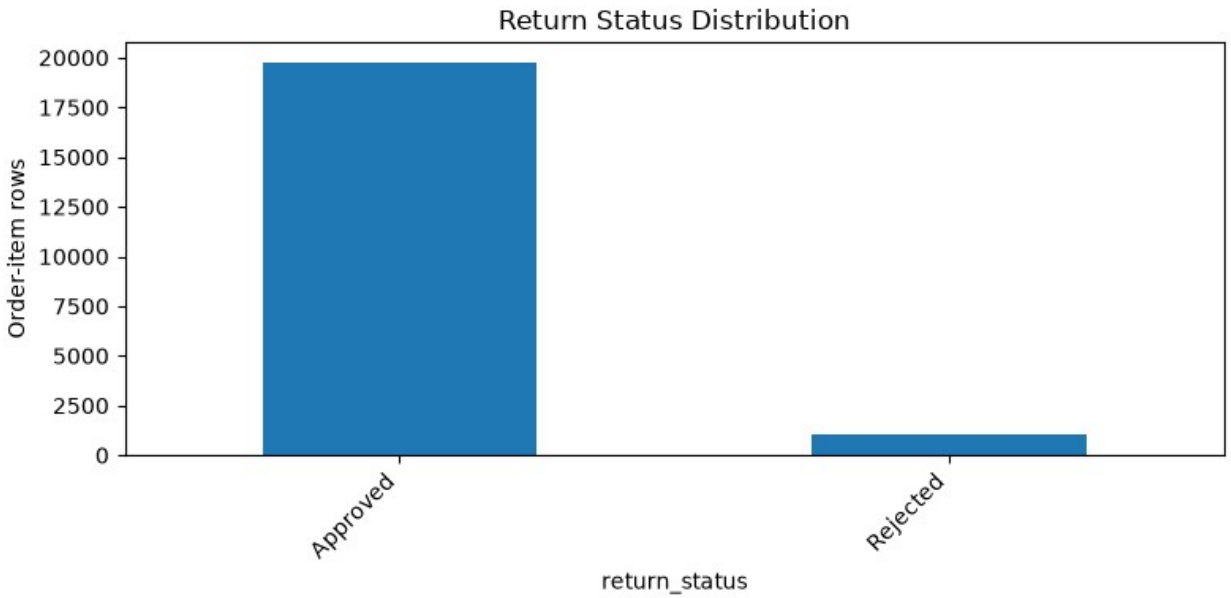


return_reason	Order-item Rows
Quality not as expected	4013
Size too small	3194
Size too large	2208
Changed my mind	2168

Arrived late	2038
Damaged product	1947
Defective / not working	1633
Colour different from image	1364
Wrong item delivered	1315
Better price elsewhere	488
Missing parts or accessories	475



return_status	Order-item Rows
Approved	19771
Rejected	1072




```
Rows with a recorded refund amount: 20843
Total recorded refund amount: 43397978.0

                            Rows
refund_mode
Original Payment Method  14767
Bank Transfer             4236
Store Credit              1840

Average recorded refund days: 4.99
```

10. Delivery and Fulfilment Analysis

```
# Compare promised vs actual delivery days
if {"promised_delivery_days",
"actual_delivery_days"}.issubset(df.columns):
    delivery = df[["promised_delivery_days",
"actual_delivery_days"]].dropna()
    if not delivery.empty:
        display(delivery.describe().round(2))
        plt.figure(figsize=(7, 5))
        sns.scatterplot(data=delivery, x="promised_delivery_days",
y="actual_delivery_days", alpha=0.35)
        max_day = max(delivery.max().max(), 1)
        plt.plot([0, max_day], [0, max_day], linestyle="--")
        plt.title("Promised vs. Actual Delivery Days")
        plt.tight_layout()
        plt.show()

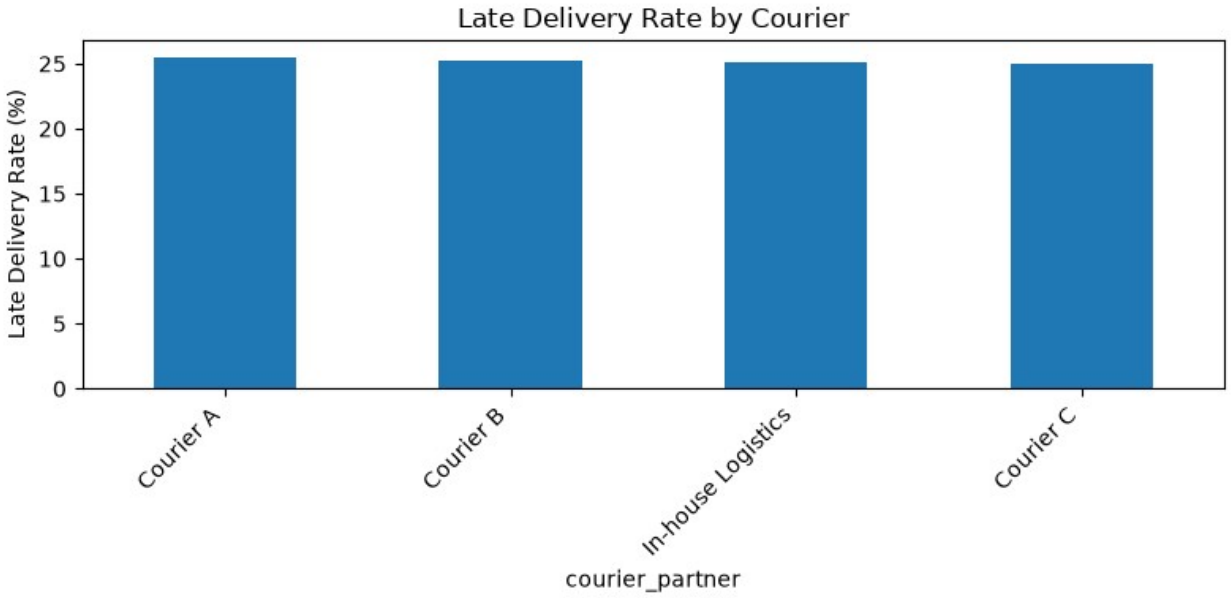
# Late delivery rate by courier; only known flags in denominator
if {"courier_partner", "is_late_delivery"}.issubset(df.columns):
```

```
    known = df[df["is_late_delivery"].isin([0, 1])]
    if not known.empty:
        courier_late = (known.groupby("courier_partner", dropna=False)
["is_late_delivery"].mean() * 100).sort_values(ascending=False)
        display(courier_late.to_frame("Late Delivery Rate (%)"))
        courier_late.plot(kind="bar", figsize=(8, 4), title="Late
Delivery Rate by Courier")
        plt.ylabel("Late Delivery Rate (%)")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()
```

	promised_delivery_days	actual_delivery_days
count	156304.00	156304.00
mean	2.57	2.79
std	1.14	1.36
min	1.00	1.00
25%	2.00	2.00
50%	2.00	2.00
75%	3.00	3.00
max	5.00	10.00



```
                      Late Delivery Rate (%)
courier_partner
Courier A                          25.503484
Courier B                          25.231678
In-house Logistics                 25.157676
Courier C                          24.966817
```



11. Monthly Trend Analysis

```
if "order_datetime" in df.columns:
    monthly_df = df.dropna(subset=["order_datetime"]).copy()
    monthly_df["order_month"] =
monthly_df["order_datetime"].dt.to_period("M").dt.to_timestamp()

    monthly_sales = monthly_df.groupby("order_month")
["item_total"].sum(min_count=1) if "item_total" in monthly_df.columns
else pd.Series(dtype=float)
    monthly_orders = monthly_df.groupby("order_month")
["order_id"].nunique() if "order_id" in monthly_df.columns else
pd.Series(dtype=float)

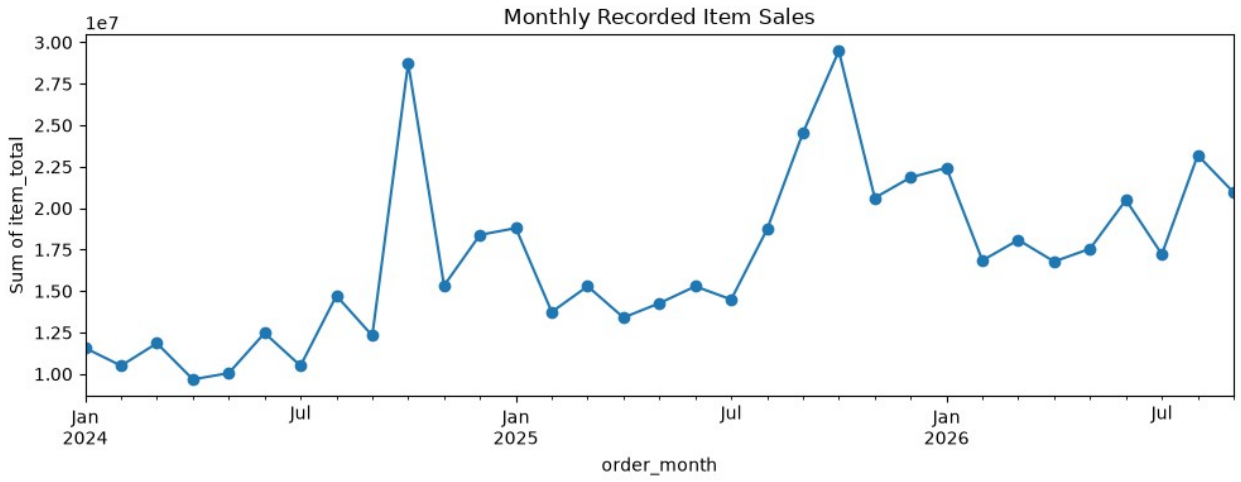
    if not monthly_sales.empty:
        monthly_sales.plot(figsize=(10, 4), marker="o", title="Monthly
Recorded Item Sales")
        plt.ylabel("Sum of item_total")
        plt.tight_layout()
        plt.show()

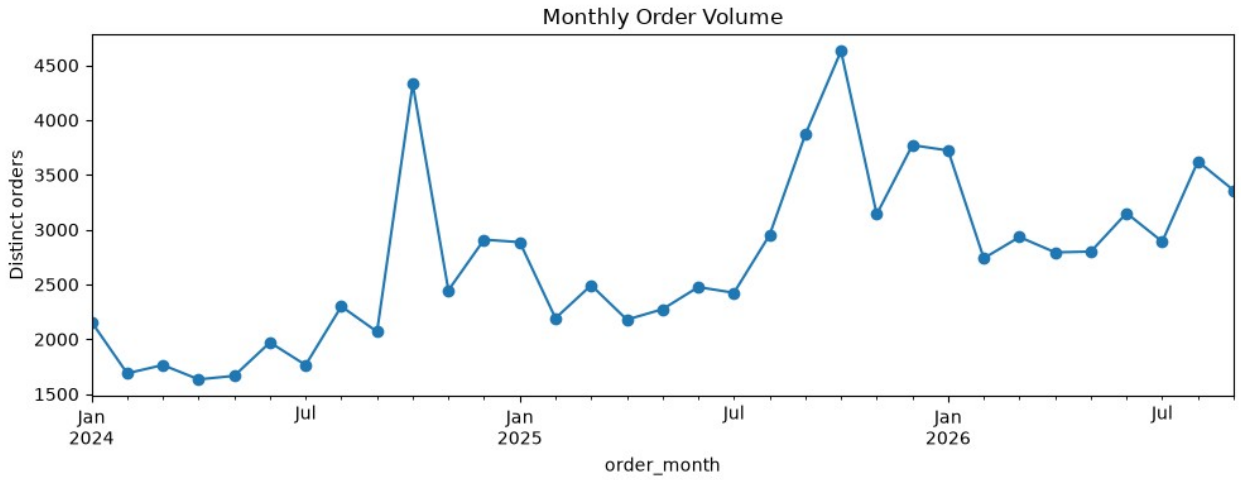
    if not monthly_orders.empty:
        monthly_orders.plot(figsize=(10, 4), marker="o",
title="Monthly Order Volume")
```

```
        plt.ylabel("Distinct orders")
        plt.tight_layout()
        plt.show()

    if "is_returned" in monthly_df.columns:
        known_returns = monthly_df[monthly_df["is_returned"].isin([0,
1])]
        if not known_returns.empty:
            monthly_return_rate = known_returns.groupby("order_month")
["is_returned"].mean() * 100
            monthly_return_rate.plot(figsize=(10, 4), marker="o",
title="Monthly Return Rate")
            plt.ylabel("Return Rate (%)")
            plt.tight_layout()
            plt.show()

    if "is_late_delivery" in monthly_df.columns:
        known_late =
monthly_df[monthly_df["is_late_delivery"].isin([0, 1])]
        if not known_late.empty:
            monthly_late_rate = known_late.groupby("order_month")
["is_late_delivery"].mean() * 100
            monthly_late_rate.plot(figsize=(10, 4), marker="o",
title="Monthly Late Delivery Rate")
            plt.ylabel("Late Delivery Rate (%)")
            plt.tight_layout()
            plt.show()
else:
    print("order_datetime is unavailable; monthly trends cannot be
calculated.")
```









12. Key Business Insights and Recommendations

```
findings = []

if {"category", "item_total"}.issubset(df.columns):
```

```
    sales_by_category = df.groupby("category")
["item_total"].sum(min_count=1).dropna()
    if not sales_by_category.empty:
        top_category = sales_by_category.idxmax()
        findings.append(
            f"Highest recorded item sales category: {top_category} "
            f"({sales_by_category.max():,.2f})."
        )

if {"category", "is_returned"}.issubset(df.columns):
    known_returns = df[df["is_returned"].isin([0, 1])]
    if not known_returns.empty:
        rates = known_returns.groupby("category")
["is_returned"].mean().dropna() * 100
        if not rates.empty:
            highest_return_category = rates.idxmax()
            findings.append(
                f"Highest observed return-rate category:
{highest_return_category} "
                f"({rates.max():.1f}%). Check category sample sizes
before acting."
            )

if {"return_reason", "is_returned"}.issubset(df.columns):
    reasons = df.loc[df["is_returned"].eq(1),
"return_reason"].dropna().value_counts()
    if not reasons.empty:
        findings.append(
            f"Most common recorded return reason: {reasons.index[0]} "
            f"({int(reasons.iloc[0]):,} item rows)."
        )

if {"courier_partner", "is_late_delivery"}.issubset(df.columns):
    known_late = df[df["is_late_delivery"].isin([0, 1])]
    if not known_late.empty:
        rates = known_late.groupby("courier_partner")
["is_late_delivery"].mean().dropna() * 100
        if not rates.empty:
            findings.append(
                f"Highest observed late-delivery rate among couriers:
{rates.idxmax()} "
                f"({rates.max():.1f}%). Review valid-record counts
before comparison."
            )

if findings:
    for number, finding in enumerate(findings, start=1):
        print(f"{number}. {finding}")
else:
    print("No findings could be generated from the available columns
```

```
and valid records.")

print("\nRecommendations should be based on these results, record
counts, and business context. "
      "Associations do not by themselves prove causes.")
```

```
1. Highest recorded item sales category: Mobiles (251,238,157.00).
2. Highest observed return-rate category: Fashion - Women (25.4%).
Check category sample sizes before acting.
3. Most common recorded return reason: Quality not as expected (4,013
item rows).
4. Highest observed late-delivery rate among couriers: Courier A
(25.5%). Review valid-record counts before comparison.

Recommendations should be based on these results, record counts, and
business context. Associations do not by themselves prove causes.
```